



IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 10: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Elabore el código I

Elabore, utilizando el Assembly RISC-V, un programa que a partir de un arreglo `arr` de largo `len`, determine si este posee un par de números **en posiciones distintas** tal que su suma sea igual a un valor `pair_sum`. Si se encuentra un par de números que cumple lo pedido, se deben guardar los índices del par en las variables `x_pair_sum` e `y_pair_sum`; en otro caso, sus valores deben ser iguales a -1. Si existe más de un par de valores que cumple lo pedido, puede escoger arbitrariamente los primeros que encuentre.

A continuación, tres ejemplos:

`arr = [1, 3, 5, 7, -5]` `pair_sum = 10` $\rightarrow$ `(x_pair_sum, y_pair_sum) = (1, 3)`
`arr = [1, 3, 5, 7, -5]` `pair_sum = 11` $\rightarrow$ `(x_pair_sum, y_pair_sum) = (-1, -1)`
`arr = [1, 3, 5, 7, -5]` `pair_sum = 14` $\rightarrow$ `(x_pair_sum, y_pair_sum) = (-1, -1)`

Puede utilizar el siguiente fragmento de código como base:

```
.data
arr:      .word 1, 3, 5, 7, -5
len:      .word 5
pair_sum: .word 10
x_pair_sum: .word -1
y_pair_sum: .word -1
.text
# Su código aquí.
```

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_x_n`:

```
.data
x:      .word 7
n:      .word 3
result: .word 0
.text
lw a0, x
lw a1, n
la s0, result
addi s1, zero, 1
addi sp, sp, -4
sw ra, 0(sp)
jal ra, func_x_n
lw ra, 0(sp)
addi sp, sp, 4
sw a0, 0(s0)
li a7, 10
ecall
func_x_n:
beq a1, s1, func_x_n_end
addi sp, sp, -8
sw ra, 0(sp)
sw a0, 4(sp)
addi a1, a1, -1
jal ra, func_x_n
lw ra, 0(sp)
lw t0, 4(sp)
addi sp, sp, 8
mul a0, a0, t0
func_x_n_end:
jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(x, n)$. A partir de este:

1. Indique, con argumentos y en términos de x y n , lo que retorna la función $f(x, n)$. Por ejemplo, $f(x, n) = x + n$.
2. Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Pregunta 3: Elabore el código II

Estamos en el año 2084. Eres el ingeniero principal de la Resistencia y has recibido un cargamento de suministros médicos vitales. El sistema de transporte automático ha dejado las cajas en el muelle de carga, pero están completamente desordenadas según su número de prioridad.

Para que los drones de distribución puedan recoger las cajas de forma eficiente, el protocolo exige que los suministros estén ordenados de **menor a mayor prioridad** en la memoria central del sistema. Debido a que el procesador del almacén es un modelo antiguo basado en la arquitectura **RISC-V** y que por la antigüedad se ha estropeado una parte del procesador, dando error al ejecutar loops, deberas programar una rutina totalmente recursiva.

El Algoritmo: Bubble Sort con Doble Recursión

En esta versión avanzada para el procesador del almacén, utilizaremos una **doble recursión** para garantizar que el sistema opere de manera puramente funcional. El problema se divide en dos procesos recursivos anidados:

1. **Recursión Externa (Control de Pasadas):** Esta función se encarga de determinar cuántas veces debemos revisar el arreglo.
 - **Caso Base:** Si el tamaño del fragmento a ordenar es $n \leq 1$, el proceso termina.
 - **Llamada Recursiva:** En cada paso, invoca a la función de la pasada interna y luego se llama a sí misma con un tamaño de $n - 1$.
2. **Recursión Interna (Pasada de Comparación):** Sustituye al bucle de recorrido tradicional. Se encarga de comparar el elemento en el índice i con el de la posición $i + 1$.
 - **Caso Base:** Si el índice i alcanza el límite superior ($n - 1$), la pasada termina y el control regresa a la función externa.
 - **Acción:** Si $arr[i] > arr[i + 1]$, se realiza el intercambio en memoria.
 - **Llamada Recursiva:** La función se llama a sí misma incrementando el índice ($i + 1$) para comparar la siguiente pareja.

Esta arquitectura permite que el procesador gestione el ordenamiento exclusivamente mediante la manipulación del **Stack Pointer (sp)** y el registro de retorno (**ra**).

Código Base

Utiliza la siguiente estructura de memoria para comenzar tu implementación:

```
.data
arr:    .word 64, 34, 25, 12, 22, 11, 90  # Prioridades de las cajas
size:   .word 7                          # Cantidad de cajas
.text
main:
    # Su código aquí
```

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

